

[2026/05/12 Euron DL session Week10 요약본]-이시현

자연어 처리(NLP)의 첫 단계인 **토큰화**는 텍스트를 의미 있는 단위로 분리하는 과정으로, 띄어쓰기, 어휘 사전, 정규 표현식, 머신러닝 등 다양한 방식으로 수행됩니다. 특히 한국어의 경우, 띄어쓰기만으로는 부족하여 형태소 분석이나 하위 단어 토큰화 같은 더 정교한 방법이 필요합니다.

1. 자연어 처리(NLP)의 첫 단계: 토큰화

토큰화는 텍스트를 의미 있는 단위로 분리하는 자연어 처리의 첫 단계이다.

1.1. 토큰화의 개념과 중요성

1. 토큰화(Tokenization)란?

1. 텍스트를 컴퓨터가 이해할 수 있도록 의미 있는 단위인 **토큰(Token)**으로 분리하는 과정이다.
2. 토큰화는 자연어 처리(NLP)의 가장 첫 번째 단계이며, 분석 대상이 되는 텍스트 데이터셋을 **코퍼스(Corpus)**라고 한다.
3. 토큰화 방식에 따라 NLP 결과가 크게 달라질 수 있다.

2. 토큰화의 필요성

1. 인간의 언어는 **모호성**과 **가변성**이라는 특성을 가지므로, 컴퓨터가 이해하기 어렵다.
2. 컴퓨터는 텍스트를 **숫자 벡터**로 변환해야만 처리할 수 있으며, 토큰화는 이 변환의 첫 단계이다.

1.2. 다양한 토큰나이저 방법

1. 띄어쓰기 분할

1. 가장 간단한 방법으로, 띄어쓰기를 기준으로 텍스트를 분리한다.
2. 한국어와 중국어처럼 띄어쓰기만으로는 부족한 언어에는 부적합하다.

2. 정규 표현식 기반 분할

1. 규칙을 사용하여 텍스트의 특정 패턴을 토큰으로 분리한다.
2. 특수 문자나 URL과 같이 구조화된 텍스트 처리에 유용하다.

3. 어휘 사전 기반 분할

1. 사전에 등록된 단어만 토큰으로 인식하는 방식이다.
2. 사전에 없는 단어(OOV, Out-Of-Vocabulary)가 발생하면 처리할 수 없는 문제가 생긴다.

4. 머신러닝 기반 토큰화

1. 데이터로부터 토큰화 규칙을 자동으로 학습하는 방식이다.
2. WordPiece, BPE(Byte Pair Encoding) 등이 대표적인 예이다.

1.3. 토큰화의 문제점: OOV와 Sparse Data

1. OOV (Out-Of-Vocabulary) 문제

1. 학습 데이터에 없던 단어가 입력으로 들어오는 문제이다.
2. 어휘 사전의 크기가 커질수록 차원의 저주(Curse of Dimensionality) 문제가 발생할 수 있다.

2. Sparse Data 문제

1. 데이터의 대부분이 0으로 채워져 있어 비효율적인 표현이 되는 문제이다.

3. 해결 방안: 하위 단어(Subword) 토큰화

1. OOV 문제를 해결하고, 단어 사전 크기를 줄여 학습 효율을 높인다.
2. 처음 보는 단어도 더 작은 조각(하위 단어)으로 분해하여 처리한다.

2. 한국어 토큰화의 특징과 방법

한국어는 띄어쓰기만으로는 부족하여 더 정교한 토큰화 방법이 필요하다.

2.1. 한국어 토큰화의 개념

1. 단어 토큰화

1. 텍스트 데이터를 의미 있는 단위인 **단어**로 분리하는 전처리 작업이다.
2. 한국어는 교착어이기 때문에, 띄어쓰기만으로는 단어 분리가 어렵다.

2. 글자(자모) 토큰화

1. 한글을 초성, 중성, 종성으로 분리하는 방식이다.
2. 장점: 작은 어휘 사전 크기, OOV 감소
3. 단점: 의미 정보 부족

3. 형태소 토큰화

1. 문법적, 의미적 최소 단위인 **형태소** 단위로 텍스트를 분리한다.
2. 한국어는 교착어 특성상 형태소 분석이 중요하다.
3. **자립 형태소**와 **의존 형태소**로 나뉜다.

2.2. 한국어 형태소 분석 라이브러리

1. KoNLPy

1. 다양한 한국어 형태소 분석기를 지원하는 라이브러리이다.
2. **Okt (Open Korean Text):**
 - SNS 기반 데이터로 학습되었다.

- 19개의 품사를 지원한다.

3. 꼬꼬마 (Kkma):

- 자바 개발 키트 기반이며, 세종 말뭉치를 사용한다.
- 21개의 품사를 지원하며, 더 자세한 분석이 가능하다.
- 분석 시간이 오래 걸리고, 모델 성능이 저하될 수 있다.

2. 라이브러리 사용 예시

1. `kkma.nouns(sentence)`: 문장에서 명사를 추출한다.
2. `okt.morphs(sentence)`: 문장을 형태소 단위로 분리한다.
3. `kkma.pos(sentence)`: 형태소와 해당 품사를 태깅한다.

2.3. NLTK를 이용한 영어 토큰화

1. NLTK (Natural Language Toolkit)

1. 자연어 처리 교육 및 연구 목적으로 사용되는 라이브러리이다.
2. 토큰화, 형태소 분석, 구문 분석, 개체명 인식 등 다양한 기능을 제공한다.

2. NLTK 토큰화 과정

1. `sent_tokenize(sentence)`: 문장 단위로 분리한다.
2. `word_tokenize(sentence)`: 단어 단위로 분리한다.
3. `pos_tag(word_tokens)`: 단어에 품사를 태깅한다.

3. 하위 단어 토큰화 (Subword Tokenization)

하위 단어 토큰화는 OOV 문제와 어휘 사전 크기 문제를 해결하는 효과적인 방법이다.

3.1. 하위 단어 토큰화의 필요성

1. 기존 형태소 분석기의 문제점

1. 신조어, 오타자, 새로운 단어 등장에 취약하다.
2. 모르는 단어를 적절한 단위로 나누는 데 어려움이 있다.
3. 어휘 사전의 크기가 커져 학습 효율이 떨어진다.

2. 하위 단어 토큰화의 해결책

1. 하나의 단어를 더 작은 하위 단어의 조합으로 나누어 토큰화한다.
2. OOV 문제, 어휘 사전 크기, 처리 속도 문제를 완화한다.
3. 신조어, 은어, 고유명사 문제에 효과적이다.
4. **인코딩 방식**: 유니그램, 워드피스, 바이트 페어 인코딩(BPE) 등이 있다.

3.2. 바이트 페어 인코딩 (BPE)

1. BPE의 원리

1. 가장 빈번하게 등장하는 연속된 글자 쌍을 탐색하여 하나의 토큰으로 조합한다.
2. 더 이상 나타나지 않거나 정해진 사전 크기에 도달할 때까지 이 과정을 반복한다.
3. 자주 등장하는 단어는 하나의 토큰으로, 덜 등장하는 단어는 여러 토큰의 조합으로 표현된다.

2. BPE 예시

1. 원문: abracadabra
2. Step #1: 가장 빈번한 글자 쌍 탐색 및 병합
3. Step #2: 새로운 빈번한 글자 쌍 탐색 및 병합

3. SentencePiece

1. 구글에서 개발한 오픈소스 하위 토큰라이저 라이브러리이다.
2. BPE와 유사한 알고리즘을 사용하며, 사용자가 직접 하이퍼파라미터를 설정할 수 있다.
3. **Korpora**를 사용하여 한국어 말뭉치를 다운로드하고 학습할 수 있다.
4. SentencePieceTrainer 클래스를 사용하여 토큰라이저 모델을 학습시킨다.
5. SentencePieceProcessor 클래스로 학습된 모델을 불러와 토큰화를 수행한다.
 - encode_as_pieces(sentence): 문장을 하위 단어 토큰으로 변환한다.
 - encode_as_ids(sentence): 토큰을 정수 ID로 변환한다.
 - decode_ids(ids): 정수 ID를 다시 문장으로 변환한다.
 - decode_pieces(pieces): 하위 단어 토큰을 문장으로 변환한다.
6. get_piece_size(): 생성된 하위 단어의 개수를 반환한다.
7. id_to_piece(idx): 정수 ID를 하위 단어로 변환한다.

3.3. 워드피스 (WordPiece)

1. 워드피스의 원리

1. BPE와 유사하지만, 빈도 기반이 아닌 **확률 기반**으로 글자 쌍을 병합한다.
2. 새로운 하위 단어를 생성할 때, 이전 하위 단어와 함께 확률을 계산하여 가장 높은 확률을 가진 쌍을 선택한다.
3. **점수 함수**: 병합하려는 하위 단어 쌍 (x, y) 의 빈도 $F(x, y)$ 를 $f(x)$ 와 $f(y)$ 로 나누어 계산한다.

2. 워드피스 예시

1. 빈번하게 등장하는 글자 쌍을 기준으로 병합을 반복한다.

2. 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 학습을 진행한다.

3. 워드피스 구현 (Tokenizers 라이브러리)

1. tokenizers 라이브러리의 WordPiece API를 사용한다.
2. 텍스트를 일관된 표준화 형식으로 정규화하는 기능을 제공한다.
3. **정규화 모듈**: NFD, 소문자 변환, 공백 처리 등을 수행한다.
4. WordPiece 토큰라이저를 학습시키고 JSON 파일로 저장한다.
5. WordPieceDecoder를 사용하여 학습된 모델을 불러와 토큰화를 수행한다.
 - encode(sentence): 문장을 토큰화한다.
 - encode_batch(sentences): 여러 문장을 한 번에 배치 인코딩한다.
 - tokens: 토큰화된 데이터의 토큰 속성을 확인한다.
 - ids: 토큰화된 데이터의 정수 ID를 확인한다.
 - decode(ids): 정수 ID를 문장으로 변환한다.

4. 텍스트 임베딩과 언어 모델

텍스트 임베딩은 단어를 벡터로 표현하고, 언어 모델은 문맥을 이해하여 다음 단어를 예측한다.

4.1. 워드 임베딩 (Word Embedding)

1. 텍스트 벡터화 방법

1. **원-핫 인코딩**: 각 단어에 고유 번호를 부여하고 해당 번호 위치만 1로 표시한다.
 - 단점: 벡터의 희소성이 크고, 단어 간 유사성을 표현하지 못한다.
2. **빈도 벡터화**: 단어의 빈도를 벡터로 표시한다.
 - 단점: 희소성이 크고, 문맥을 고려하지 못한다.

2. 워드 임베딩

1. 단어를 **밀집 벡터(Dense Vector)**로 표현하는 방법이다.
2. 의미가 비슷한 단어들은 벡터 공간에서 비슷한 위치에 표현된다.
3. **Word2Vec, fastText** 등이 대표적인 모델이다.

3. 동적 임베딩

1. 단어가 문맥에 따라 뜻이 달라지는 것을 해결하기 위해 사용된다.

4.2. 언어 모델 (Language Model)

1. 언어 모델의 역할

1. 앞에 나온 단어들을 바탕으로 다음에 올 단어를 예측하는 모델이다.
2. 문장의 흐름과 문맥을 이해하여 자연스러운 문장을 생성한다.

2. 자기회귀 언어 모델

1. 앞에 나온 단어들을 순서대로 예측하여 다음 단어를 모델링한다.
2. 이전 단어들의 정보를 이용해 다음에 나올 단어의 확률을 계산한다.
3. **조건부 확률**: $P(w_i | w_1, w_2, \dots, w_{i-1})$

3. 통계적 언어 모델

1. 이전에 나온 단어들의 패턴과 확률을 이용해 다음 단어를 예측한다.
2. 주로 **마르코프 체인**을 이용하여 구현된다.
3. **한계**: 긴 문맥 이해 어려움, 학습하지 않은 단어 예측 어려움

4. N-gram 모델

1. 연속된 N개의 단어를 묶어서 다음 단어를 예측하는 가장 기본적인 언어 모델이다.
2. N-1개의 이전 단어만 참고하여 다음 단어의 확률을 계산한다.
3. **종류**: 유니그램(N=1), 바이그램(N=2), 트라이그램(N=3) 등
4. **파이썬 구현**: NLTK 라이브러리의 ngrams 함수를 사용한다.

4.3. TF-IDF (Term Frequency-Inverse Document Frequency)

1. TF-IDF의 개념

1. 문서에서 단어의 중요도를 평가하는 방법이다.
2. 특정 문서에서 자주 등장하는 단어는 중요도를 높게 평가한다.
3. 여러 문서에서 흔하게 등장하는 단어는 중요도를 낮게 평가한다.

2. TF (Term Frequency)

1. 특정 단어가 특정 문서에 얼마나 자주 등장하는지를 나타낸다.

3. IDF (Inverse Document Frequency)

1. 특정 단어가 전체 문서 중 몇 개의 문서에 나타나는지를 나타낸다.
2. 적은 문서에 등장할수록 중요한 단어로 판단한다.
3. $IDF(t, D) = \log(1 + D / DF(t, D))$

4. TF-IDF 계산

1. $TF-IDF(t, d, D) = TF(t, d) * IDF(t, d)$
2. 특정 문서에서 자주 등장하면서도, 다른 문서에서는 등장하지 않는 단어의 중요도를 높게

평가한다.

5. **TfidfVectorizer (Scikit-learn)**

1. 텍스트 데이터를 TF-IDF 형태로 변환하는 클래스이다.
2. `fit_transform(corpus)`: 말뭉치를 학습하고 TF-IDF 행렬로 변환한다.
3. `tfidf_matrix`: (문서 수) x (단어 수) 형태의 넘파이 배열로 변환된다.
4. `vocabulary_`: 학습된 단어 사전을 반환한다.